



Building Automated Machine Images using **HashiCorp Packer**



Hello!

I am Bryan Krausen

I'm a Principal Solutions Architect specializing in HashiCorp and AWS solutions.

Currently have many courses on other HashiCorp products, such as Terraform, Vault, and Consul



@btkrausen



github.com/btkrausen



<https://krausen.io>

HashiCorp

AMBASSADOR
2021

*Not Affiliated with HashiCorp

Hello!

I am Gabe Maentz

I'm a Director of Solution
Architecture specializing in
HashiCorp and **DevOps** solutions

Experienced practitioner, life-long learner with a
passion for teaching. I am a lead on many private
courses on as **Terraform**, **Vault**, and **GitLab**.



@gmaentz



github.com/gmaentz



<https://maentz.net>

 HashiCorp

AMBASSADOR
2021

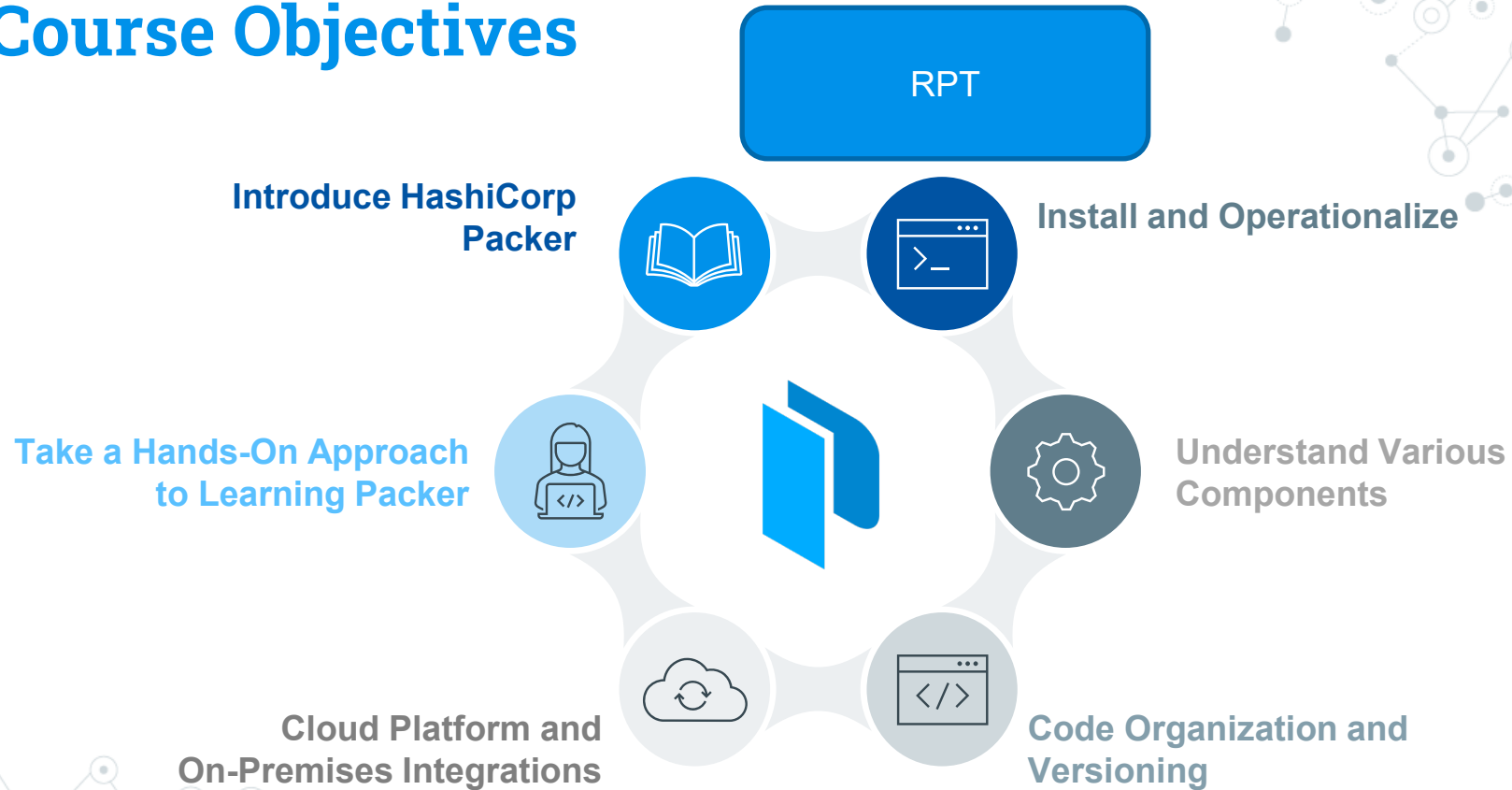
*Not Affiliated with HashiCorp

Experience

Bryan and Gabe bring
almost 10 years of combined
experience with HashiCorp
Products



Course Objectives

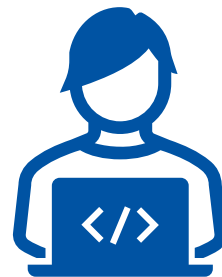


Course Format



- ✓ Provide basis for discussion and topic at hand
- ✓ Make sure students understand terms and definitions
- ✓ Will set the stage for Packer hands-on labs
- ✓ Will attempt to be short and to the point

- ✓ Hands-on discussions using one or more Packer files
- ✓ For the most part, demos will use a single cloud/platform
 - ✓ Multi-platform Hands-On Labs will also be included



Hands-On Labs



Course Content



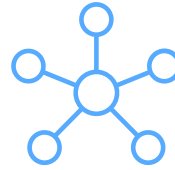
Slides

Available in
PDF in Each
Section



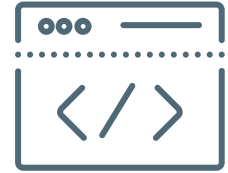
Resources

References
Links
Available in
Each Section



Mind Map

PDF of a Mind
Map for Each
Section



Lab Guides

PDF of Each
Lab
Demonstration





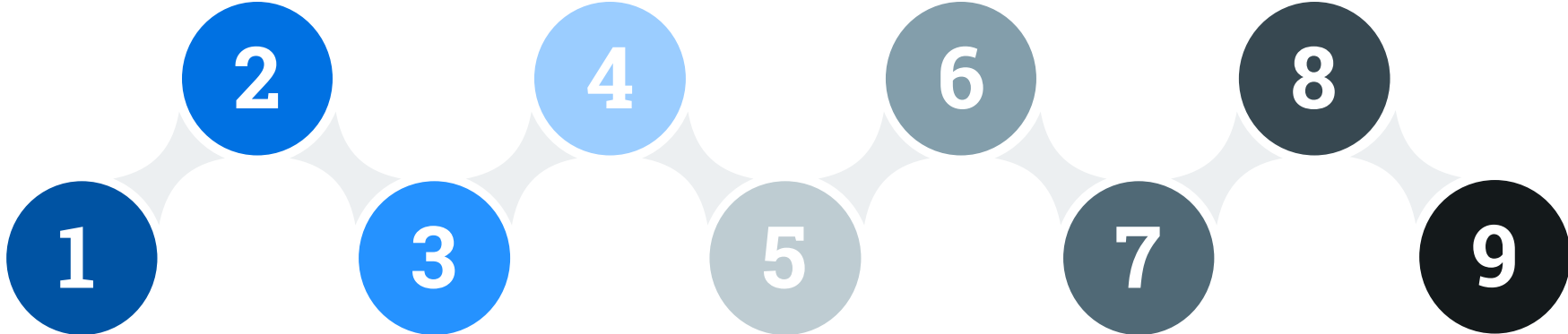
RPT

Writing Packer
Templates

Variables

Plugins

Integrations



Introduction to
HashiCorp
Packer

Builders

Provisioners

Code
Organization

Other Uses
For Packer

Course Agenda



Course Resources

RPT



github.com/btkrausen/hashicorp



**Send Us a Message or Use
the Udemy Q&A Feature**





Let's Get Started





**END OF
SECTION**





Introduction to HashiCorp Packer



Introduction to HashiCorp Packer

Section Overview

What is Packer?

Primary Use Cases

Core Components

Installing Packer

Interacting with Packer (CLI)

Packer Workflow

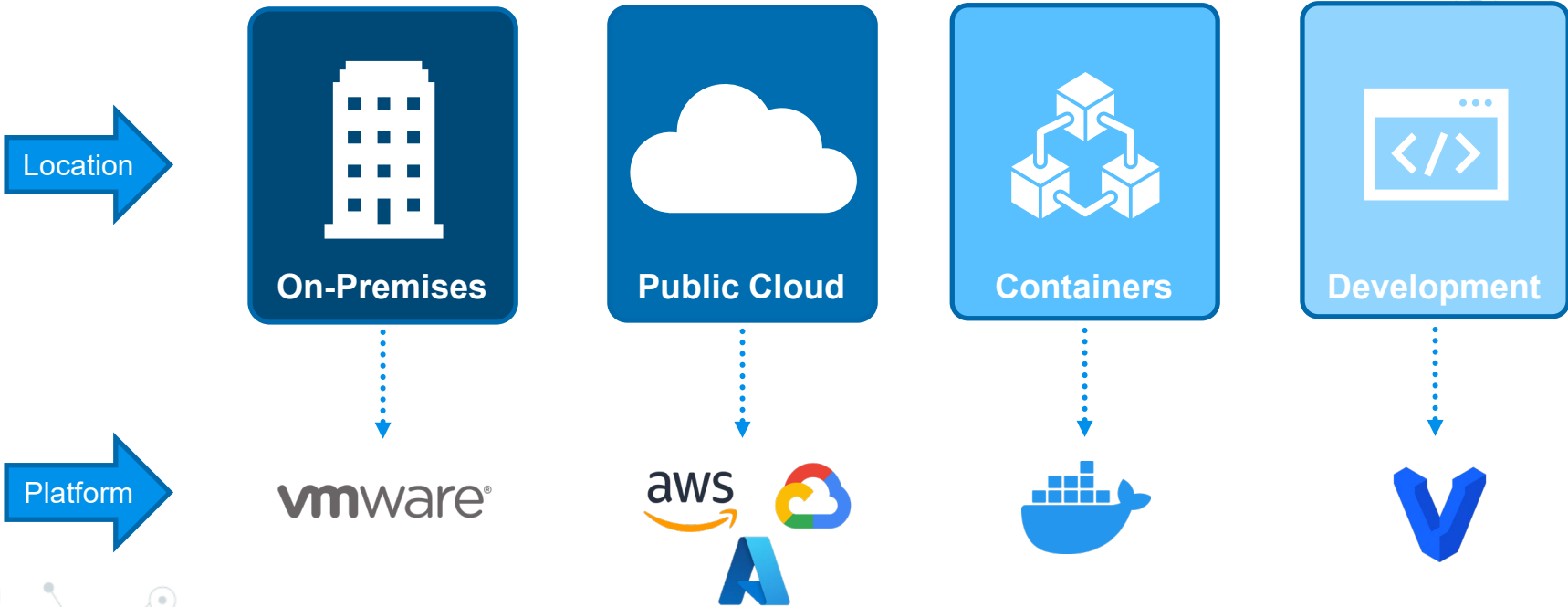




Why Use Packer?



Evolving Architecture at Enterprises

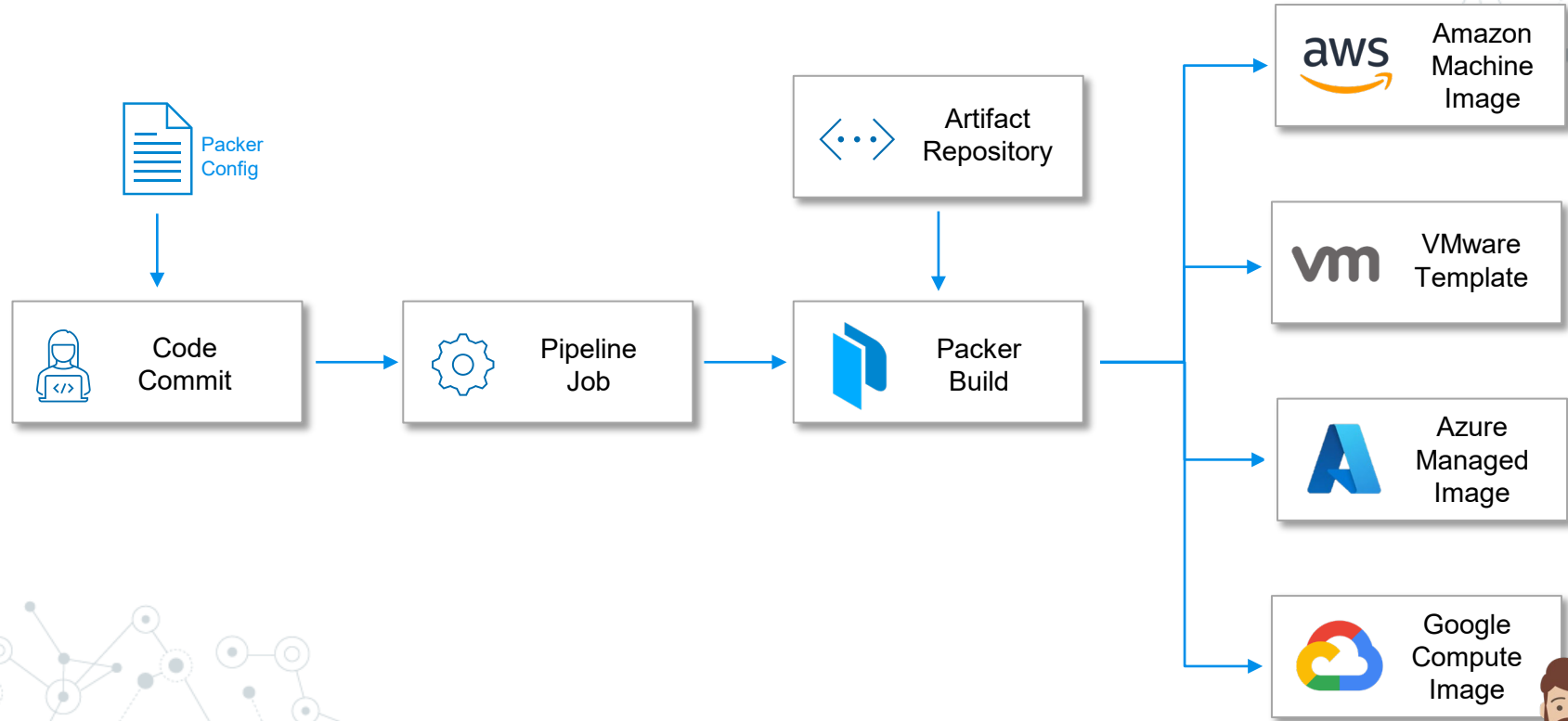


Each Platform Requires Its Own Type Of Image

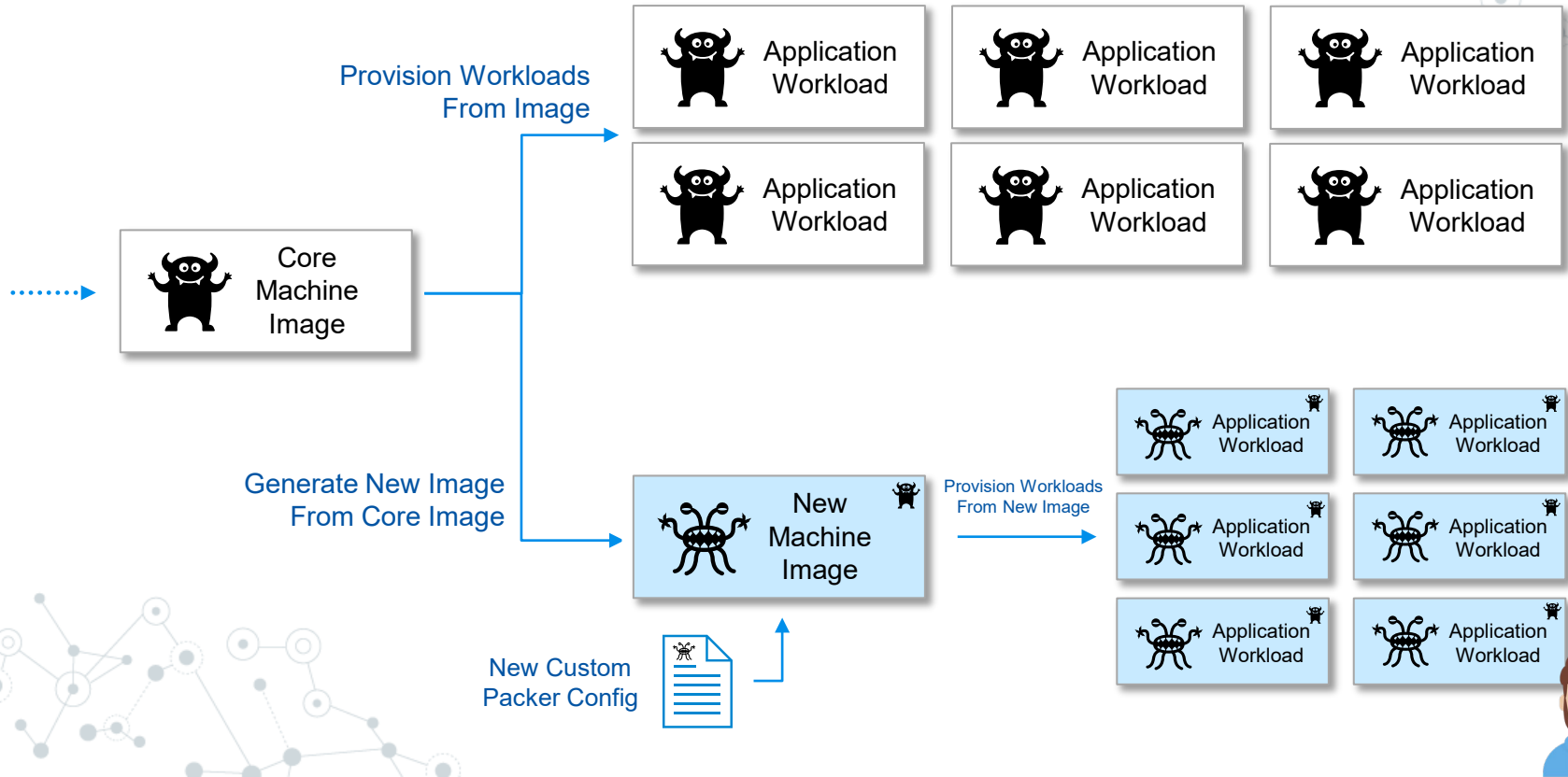


Automate Image Builds Across Platforms

Golden Images For All Your Workloads

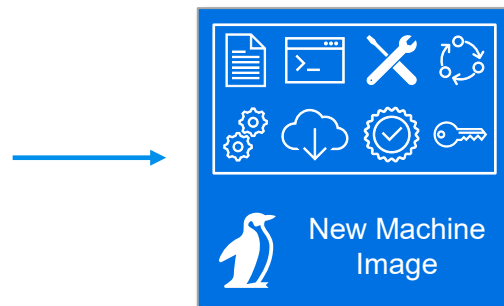
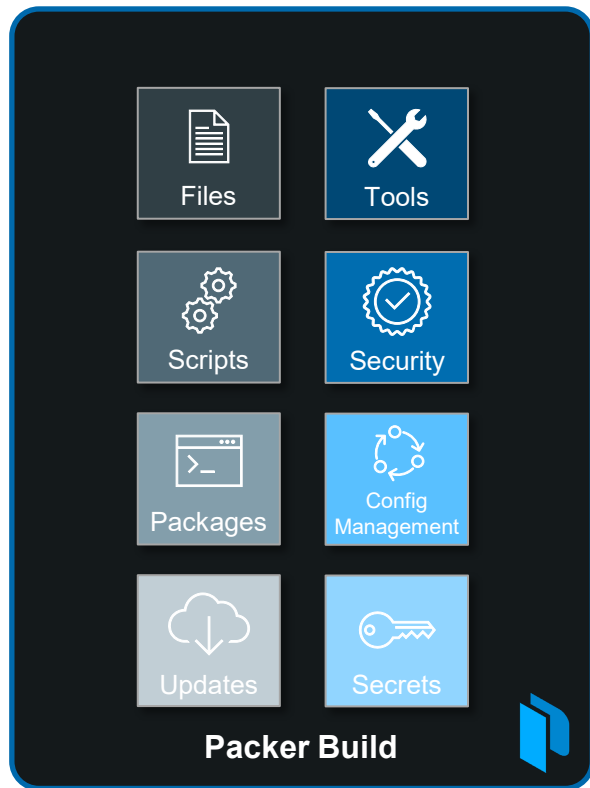
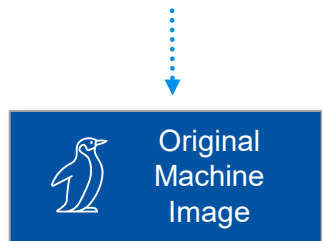


Maintain Consistency Across Workloads



Building Your Ideal Image for Workloads

Manufacturer Image (MS, RedHat, etc)
Manufacturer ISO File (Boot File, Image)
Image from Cloud Provider (Marketplace)
Custom Image Built Internally
Image Previously Built by Packer





Hello, Packer!



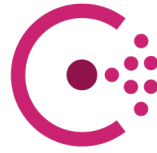
HashiCorp Suite of Tools



HashiCorp
Terraform



HashiCorp
Vault



HashiCorp
Consul



HashiCorp
Nomad



HashiCorp
Waypoint



HashiCorp
Boundary



HashiCorp
Packer



HashiCorp
Vagrant



What is HashiCorp Packer?

- ✓ Open-Source Machine Image Creation Tool
- ✓ Automates the Installation and Configuration on Packer-made Images
- ✓ Works with Multiple Platforms – Even from the Same Configuration
- ✓ Eliminates Manual Steps for Golden Image Creation



HashiCorp
Packer



vmware®



Primary Use Cases



Establish an Image Factory
Based on New Commits for
Continuous Delivery



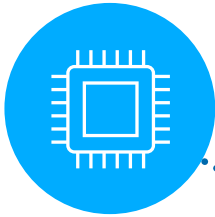
Automate Your Monthly Patching
For New/Existing Workloads



Create Immutable Infrastructure
Using Packer in CI/CD Pipeline



Create Golden Images
Across Platforms and
Environments



Benefits of Using Packer

Version Controlled

Images are Defined and Versioned as Code

Rather than the manual control of images, using Packer allows you to version control your images to simplify long-term management and updates within the organization



Consistent Images

Cross-Platform Consistency

When you use multiple platforms for workloads, maintaining like-for-like images becomes a demanding task. Packer can assist with management across multiple platforms.



Automate Everything

Stop the Manual Madness

Manually having to manage images across multiple platforms is prone to human-error and creates unneeded administration overhead.





Core Components of Packer



Core Components

- HashiCorp Packer builds images using a **template**
- Templates can be built using either JSON_(old) or HCL2_(recommended for Packer 1.7.0+)
- Template defines settings using blocks:
 - ✓ Original **Image** to Use (source)
 - ✓ **Where** to Build the Image (AWS, VMware, OpenStack)
 - ✓ **Files** to Upload to the Image (scripts, packages, certificates)
 - ✓ Installation and **Configuration** of the Machine Image
 - ✓ **Data** to Retrieve when Building



Core Components

Template Example (HCL2)



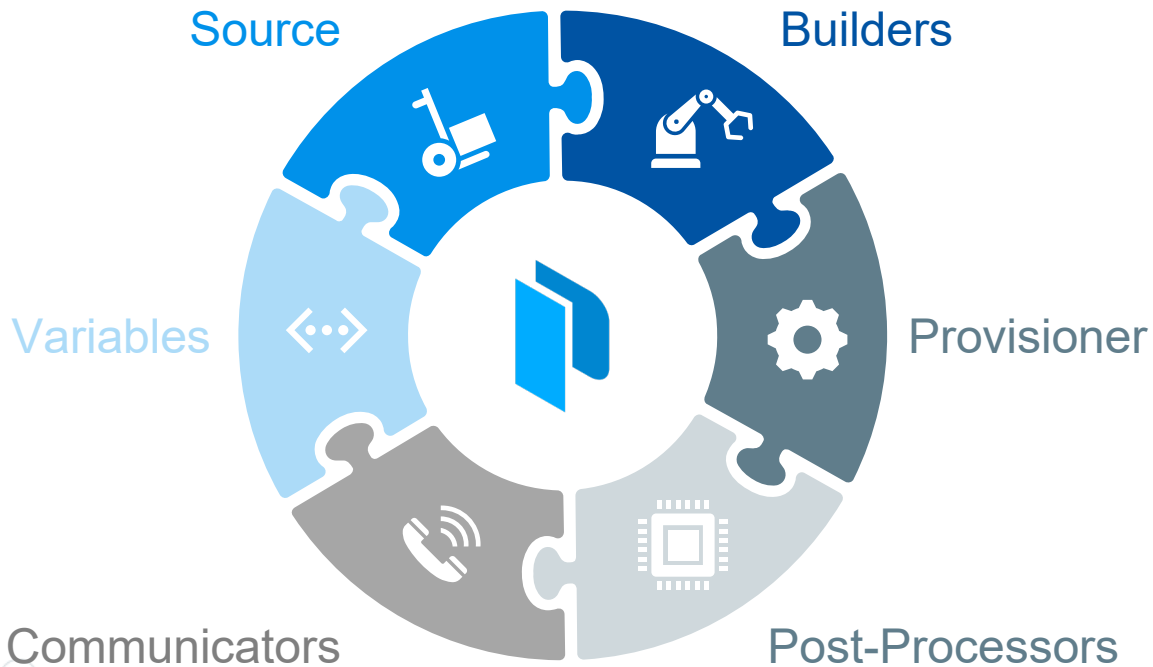
```
source "amazon-ebs" "aws-example" {
  ami_name      = "${var.ami_name}"
  instance_type = "t3.medium"
  region        = "us-east-1"
  source_ami_filter {
    filters = {
      name                = "${var.source_ami_name}"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    owners = ["amazon"]
  }
  ssh_username = "ec2-user"
  subnet_id    = "${var.subnet_id}"
  tags = {
    Name = "${var.ami_name}"
  }
  vpc_id = "vpc-1234567890"
}
```

```
build {
  sources = ["source.amazon-ebs.aws-example"]

  provisioner "file" {
    destination = "/tmp"
    source      = "files"
  }
  provisioner "shell" {
    script = "scripts/setup.sh"
  }
  provisioner "shell" {
    script = "scripts/vault.sh"
  }
}
```



Core Components



Source

- Source defines the initial image to use to create your customized image. Any defined source is reusable within build blocks
- For example:
 - Building a new AWS image (AMI), you need to point to an [existing AMI](#) to customize
 - Creating a new [vSphere template](#) requires the name of the source VM
 - Building new Google Compute images needs a [source image](#) to start

```
azure.pkr.hcl

source "azure-arm" "azure-arm-centos-7" {
  image_offer      = "CentOS"
  image_publisher  = "OpenLogic"
  image_sku        = "7.7"
  os_type          = "Linux"
  subscription_id  = "${var.azure_subscription_id}"
}
```



Builders

- Builders are responsible for **creating machines** from the base image, customizing the image as defined, and then creating a resulting image
- Builders are **plugins** that are developed to work with a specific platform (i.e., AWS, Azure, VMware, OpenStack, Docker)
- Everything done to the image is done within the **BUILD** block
- This is where the customization "work" happens

```
azure.pkr.hcl

build {
  source = ["source.azure-arm.azure-arm-centos-7"]

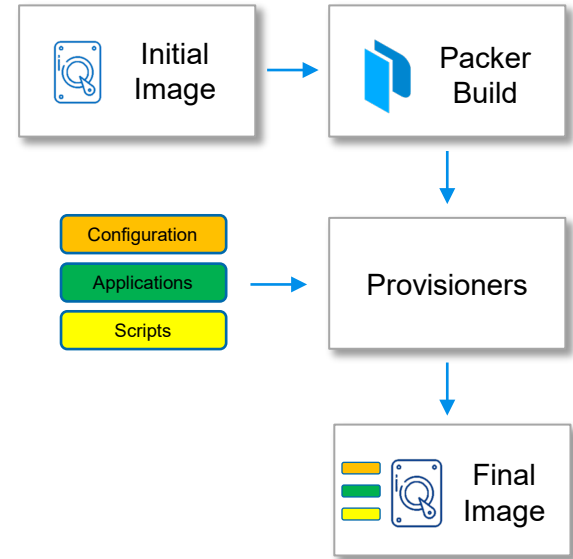
  provisioner "file" {
    destination = "/tmp/package_a.zip"
    source      = "${var.package_a_zip}"
  }
}
```

Uses the Azure builder to create a new Azure Machine Image



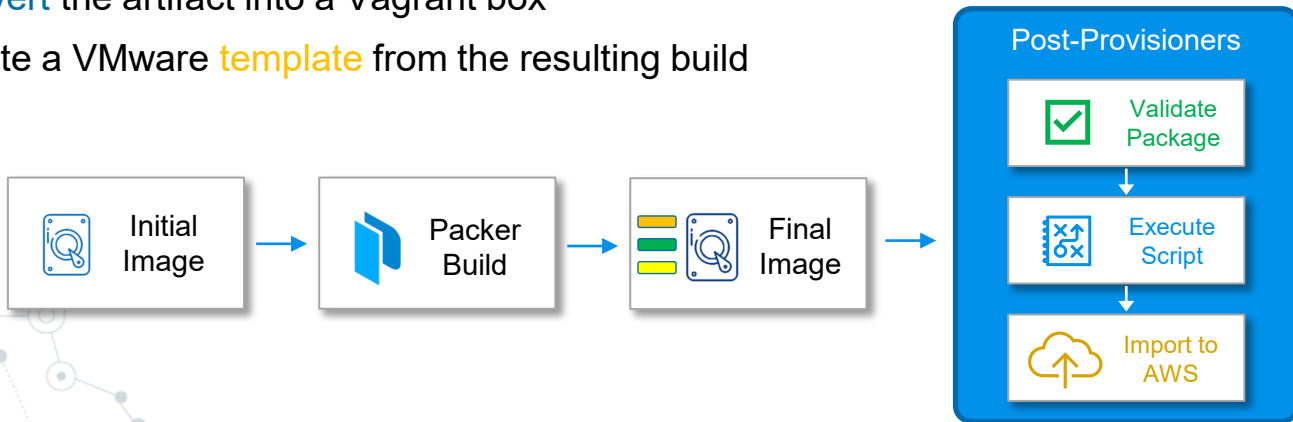
Provisioners

- Provisioners use built-in and third-party integrations to **install packages** and **configure the machine image**
- Built-in integrations include **file** and different **shell** options
- Third-party integrations include:
 - **Ansible** – run playbooks
 - **Chef** – run cookbooks
 - **InSpec** – run InSpec profiles
 - **PowerShell** – execute PowerShell scripts
 - **Puppet** – run Puppet manifest
 - **Salt** – configure based on Salt state
 - **Windows Shell** – runs commands using Windows cmd



Post-Processors

- Post-processors are executed after the image is built and provisioners are complete. It can be used to upload artifacts, execute uploaded scripts, validate installs, or import an image
- Examples include:
 - Validate a package using a [checksum](#)
 - [Import](#) a package to AWS as an Amazon Machine Image
 - [Push](#) a Docker image to a registry
 - [Convert](#) the artifact into a Vagrant box
 - Create a VMware [template](#) from the resulting build



Communicators

- Communicators are the mechanism that Packer will use to communicate with the new build and upload files, execute scripts, etc.
- Two Communicators available today:
 - SSH
 - WinRM



Variables

- HashiCorp Packer can use **variables** to define defaults during a build
- Variables can be declared in a **.pkrvars.hcl** file or **.auto.pkrvars.hcl**, the default .pkr file, or any other file name if referenced when executing the build.
- You can also declare individually using the **-var** option
- **Variable declaration block:**

```
variable "image_id" {  
  type      = string  
  description = "The id of the machine image (AMI) to use for the server."  
  default   = "ami-1234abcd"  
  
  validation {  
    condition     = length(var.image_id) > 4 && substr(var.image_id, 0, 4) == "ami-"  
    error_message = "The image_id value must be a valid AMI id, starting with \"ami-\"."  
  }  
}
```



About HashiCorp Packer



HashiCorp Packer is platform agnostic....meaning it can be installed and run on many different underlying platforms



Cloud-Based Machines (AWS Instances, Azure Virtual Machines, Google Compute)



VMware Virtual Machines



Physical Servers



Desktop/Laptop



About HashiCorp Packer



HashiCorp Packer is also available for many different operating systems:

- ✓ macOS
- ✓ Windows
- ✓ Linux
- ✓ FreeBSD
- ✓ OpenBSD
- ✓ Solaris



Using Packer

1 Install Packer

2 Create Template

3 Build Machine Image

More information in the
Writing Packer Templates
Section

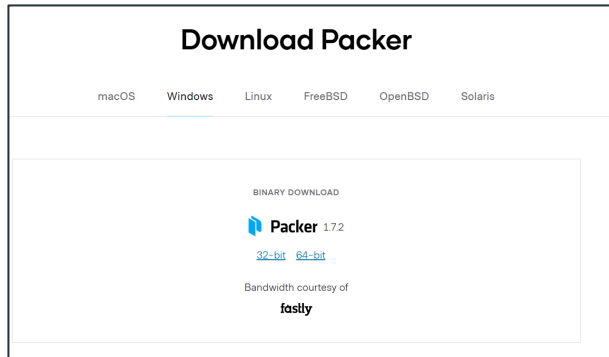
More information in this
Section and throughout the course



Install HashiCorp Packer

So where do I download Packer?

- packer.io
- releases.hashicorp.com/packer



You can also download/install Packer using your preferred package manager (apt, yum, homebrew, chocolatey)

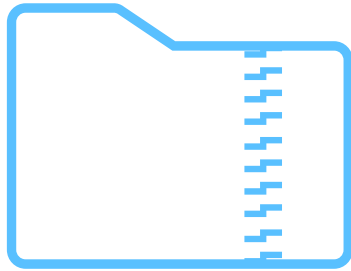
Terminal

```
$ curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo apt-key add -  
$ sudo apt-add-repository "deb [arch=amd64] https://apt.releases.hashicorp.com $(lsb_release -cs) main"  
$ sudo apt-get update && sudo apt-get install packer
```



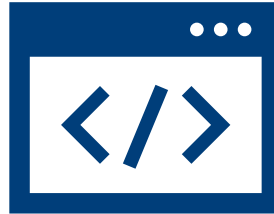
Install HashiCorp Packer

Manual Install



packer.zip

.....>
unzip



Packer binary

.....>
move

\$PATH

Set Path To Executable



Packer Command Line

The command line interface (CLI) is how users/applications interact with Packer

- There is no UI or API for Packer

Terminal

```
$ packer
```

```
Usage: packer [--version] [--help] <command> [<args>]
```

Available commands are:

build	build image(s) from template
console	creates a console for testing variable interpolation
fix	fixes templates from old versions of packer
hcl2_upgrade	transform a JSON template into an HCL2 configuration
inspect	see components of a template
validate	check that a template is valid
version	Prints the Packer version



Packer Command Line

packer autocomplete

Install Packer autocomplete to enable tab completion when using the CLI:

```
Terminal

$ packer -autocomplete-install

$ packer p
plugin build

$ packer build -
-color -debug -except -force
-machine-readable -on-error -only -parallel
-timestamp -var -var-file
```

Type p and hit <tab>



Packer Command Line

Similar to any other command-line based tool, Packer uses a subcommand and additional arguments to execute Packer functionality

Terminal

```
$ packer <sub-command> <argument> <argument>
```

All commands start with the 'packer' command

Terminal

```
$ packer build -var-file=vars.pkr.hcl aws.pkr.hcl
```

Packer
command

Sub
command

Argument
(option)

Template
Name



Packer Command Line

Subcommands available in Packer:

- `build` - build image(s) from template
- `console` - creates a console for testing variable interpolation
- `fix` - fixes templates from old versions of packer
- `hcl2_upgrade` - transform a JSON template into an HCL2 configuration
- `inspect` - see components of a template
- `validate` - check that a template is valid
- `version` - prints the Packer version

Most of the commands accept or require flags or arguments to execute the desired functionality



Packer Command Line

packer `build`

The `packer build` command takes a Packer template and runs all the defined builds to generate the desired artifacts. The `build` command provides the core functionality of Packer.

 Terminal

```
$ packer build base-image.pkr.hcl
```

Important Options/Arguments

- `-debug` – enables debug mode for step-by-step troubleshooting
- `-var` – sets a variable in the Packer template
- `-var-file` – use a separate variable file



Packer Command Line

packer `fix`

The `packer fix` command takes a template and finds backwards incompatible parts of it and brings it up to date so it can be used with the latest version of Packer. Use after you update Packer to a new release version.

JSON ONLY COMMAND

- As of Packer 1.7.2, the command is not yet available with HCL2 templates yet, as it is not yet needed.

Terminal

```
$ packer fix old-template.json > new-template.json
```



Packer Command Line

packer `fmt`

The `packer fmt` command is used to format your Packer templates and files to the preferred HCL canonical format and style.

Terminal

```
$ packer fmt base-image.pkr.hcl
```

Not Formatted



```
variable "vpc_id" {  
type=string  
default="vpc-06626bb552084b94b"  
}
```

packer fmt

Formatted Correctly



```
variable "vpc_id" {  
  type    = string  
  default = "vpc-06626bb552084b94b"  
}
```



Packer Command Line

packer inspect

The `packer inspect` shows all components of a Packer template including variables, builds, sources, provisioners and post-processors

```
Terminal
$ packer inspect aws-ubuntu.pkr.hcl
Packer Inspect: HCL2 mode

> input-variables:
> local-variables:
> builds:
  > Amazon AMI:
    sources:
      amazon-ebs.ubuntu
    provisioners:
      shell
    post-processors:
      0:
        manifest
```



Packer Command Line

packer validate

The `packer validate` will validate the syntax and the configuration of your Packer template. This is your first validation for templates after writing or updating them.

```
Terminal
$ packer validate temp.pkr.hcl
```

Template validated
(no error message)



```
Terminal
$ packer validate vmware-image.pkr.hcl
Template validation failed. Errors are shown below.

Errors validating build 'vmware'. 1 error(s) occurred:

* Either a path or inline script must be specified.
```

Template did not validate
successfully
(error shown)



Packer Command Line

packer hc12_upgrade

The `packer hc12_upgrade` translates a template written in the older JSON format to the new HCL2 format.

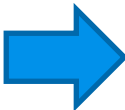
Converted JSON to
HCL2 format

Terminal

```
$ packer hc12_upgrade base-image.json  
Successfully created base-image.json.pkr.hcl
```

base-image.json

```
{  
  "variables": {  
    "deployment_version": "version-1",  
    "ami_prefix": "amzn2",  
    "app_name": "consul",  
    "consul_version": "1.9.3",  
  },  
}
```



base-image.json.pkr.hcl

```
variable "deployment_version" {  
  type    = string  
  default = "version-1"  
}  
variable "ami_prefix" {  
  type    = string  
  default = "amzn2"  
}  
variable "app_name" {  
  type    = string  
  default = "consul"  
}
```



Packer Command Line

Environment Variables

Packer has a few environment variables that you should know:

- PACKER_LOG – enable Packer detailed logs (off by default)
- PACKER_LOG_PATH – set the path for Packer logs to specific file (rather than stderr)
- PKR_VAR_xxx – define a variable value using ENV rather than in a template

Terminal

```
$ export PACKER_LOG=1
$ export PACKER_LOG_PATH=/var/log/packer.log
$ packer build base-image.pkr.hcl
```

1. Enable Detailed Logs
2. Set a path for logs
3. Run the packer build

Terminal

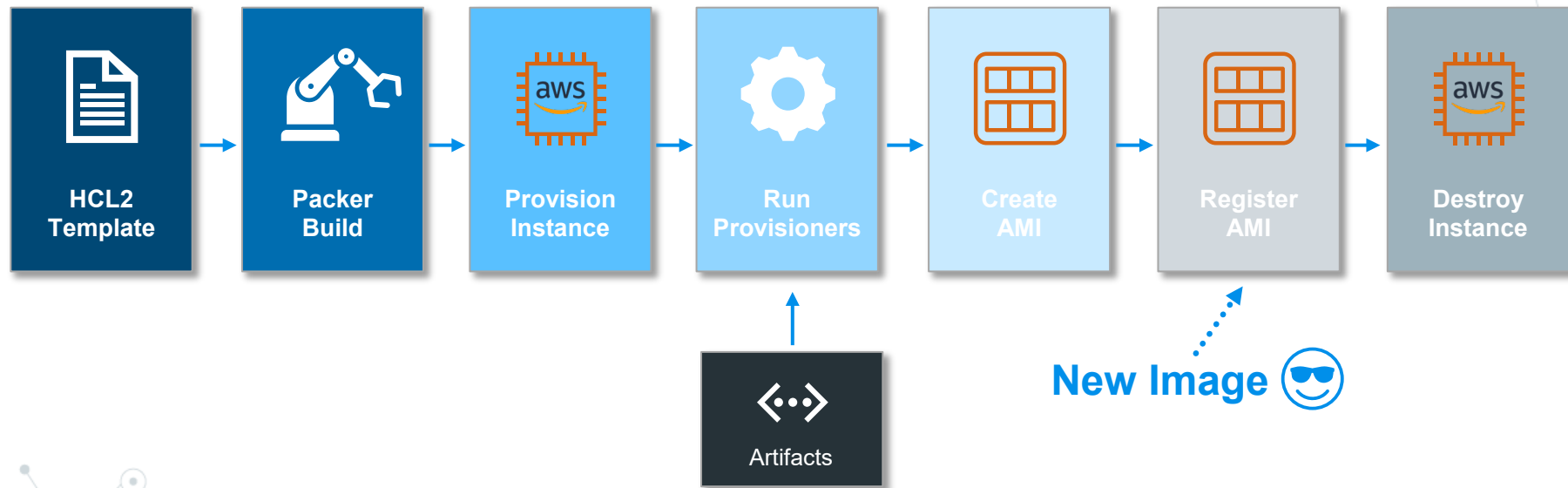
```
$ export PKR_VAR_aws_region=us-east-1
$ packer build aws-base-image.pkr.hcl
...
```

Declare a value for the aws_region variable using ENV



The Packer Workflow

AWS Example



Introduction to HashiCorp Packer

Section Recap

What is Packer?

Primary Use Cases

Core Components

Installing Packer

Interacting with Packer (CLI)

Packer Workflow





**END OF
SECTION**





Writing Packer Templates



Packer Templates

- The core functionality and behavior of [HashiCorp Packer](#) is defined by a **template**
- Templates consist of declarations and command, such as what plugins (builders, provisioners, etc.) to use, how to configure the plugins, and what order to run them

Packer templates
support two formats

HCL2

(HashiCorp Configuration Language)

{ JSON }

 This course focuses on writing Packer templates using the HCL2 format



Packer Templates

The Future of Packer is Now

HashiCorp is moving to **HCL2** as the standard format for Packer 1.7+



JSON was previously the standard format

Mighty Fine Design

HCL is designed to strike a balance between human-readable and machine-parsable



Packer templates are very easy to develop and read

Look Familiar?

If you have Terraform experience, Packer templates are going to look very familiar



HCL is the same configuration language as Terraform and other HashiCorp products

Note: JSON templates will enter a legacy support phase before being deprecated



HCL Formatting



Configuration format is VCS friendly
(multi-line lists, trailing commas, auto-formatting)

Only code blocks built into the HCL language
are available for use

Packer uses a standard file name for simplicity
`<name>pkg.hcl`

Uses Syntax Constructs like Blocks and
Arguments

New features will only be implemented for the
HCL format moving forward



HCL Syntax

```
syntax.pkr.hcl

<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
}
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
}
<BLOCK TYPE> "<BLOCK LABEL>" "<BLOCK LABEL>" {
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
  <IDENTIFIER> = <EXPRESSION>
}
```

Block

Arguments

Arguments



HCL File Structure



-
- ├── README.md
- ├── ubuntu.pkr.hcl
- ├── centos.pkr.hcl
- ├── rhel.pkr.hcl
- ├── web-app.pkr.hcl
- └── prod.pkrvars.hcl

Build all templates in a directory

Terminal

```
$ packer build /opt/packer/templates
```

Build using a single template

Terminal

```
$ packer build rhel.pkr.hcl
```



HCL Example

```
source "amazon-ebs" "aws-example" {
  ami_name      = "${var.ami_name}"
  instance_type = "t3.medium"
  region        = "us-east-1"
  source_ami_filter {
    filters = {
      name                = "${var.source_ami_name}"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    owners = ["amazon"]
  }
  ssh_username = "ec2-user"
  subnet_id    = "${var.subnet_id}"
  tags = {
    Name = "${var.ami_name}"
  }
  vpc_id = "vpc-1234567890"
}
```

```
build {
  sources = ["source.amazon-ebs.autogenerated_1"]

  provisioner "file" {
    destination = "/tmp"
    source       = "files"
  }
  provisioner "shell" {
    script = "scripts/setup.sh"
  }
  provisioner "shell" {
    script = "scripts/vault.sh"
  }
}
```



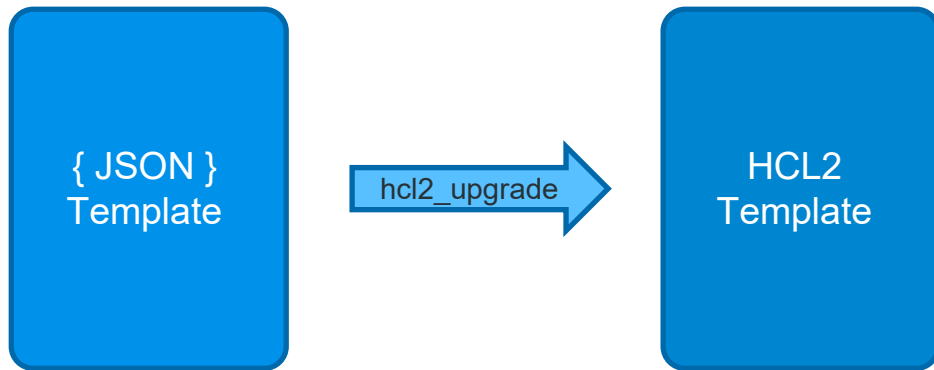
JSON Example

```
{
  "variables": {
    "ami_prefix": "amzn2",
    "ami_name": "{{user `ami_prefix`}}-{{user `app_name`}}-ent-{{user `consul_version`}}-{{timestamp}}",
    "region": "us-east-1",
    "vpc_id": "vpc-1234567890",
    "subnet_id": "subnet-1234567890",
    "ssh_username": "ec2-user",
    "source_ami_name": "amzn2-ami-hvm-2.0.20200304.0-x86_64-gp2",
    "source_ami_owner": "amazon"
  },
  "builders": [{
    "type": "amazon-ebs",
    "region": "{{user `region`}}",
    "vpc_id": "{{user `vpc_id`}}",
    "subnet_id": "{{user `subnet_id`}}",
    "source_ami_filter": {
      "filters": {
        "virtualization-type": "hvm",
        "name": "{{user `source_ami_name`}}",
        "root-device-type": "ebs"
      },
      "owners": ["{{user `source_ami_owner`}}"]
    },
    "tags": {
      "Name": "{{user `ami_name`}}"
    }
  }],
  "provisioners": [
    {
      "type": "file",
      "source": "files",
      "destination": "/tmp"
    }
  ]
}
```



Converting Old Templates to HCL

- If you have older, JSON formatted templates, you can easily convert them using the `packer hcl2_upgrade` command
- All fields of builders, provisioners and post-processors have a 1:1 correspondence with a few exceptions.



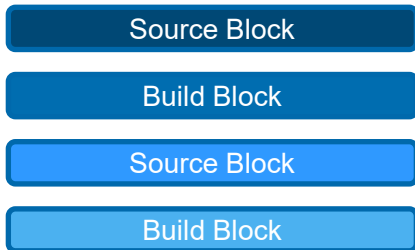


HCL Syntax



Block Organization

- In general, the ordering of root blocks is not significant within a Packer template since Packer uses a declarative model. References to other resources do not depend on the order they are defined.
- Blocks can even span multiple Packer template files.
- The order of **provisioner** or **post-processor** blocks within a build is the only major feature where block order matters.



Order of blocks within a template is not significant



Commenting in HCL2

HCL2

- HCL2 supports comment to use throughout your configuration file
 - `#` - single line comment
 - `//` - single line comment
 - `/* and */` - start and stop delimiters – might span over multiple lines

```
Terminal

# this is a comment
source "amazon-ecs" "example" {
  ami_name = "abc123"
}
// this is also a comment
```

```
Terminal

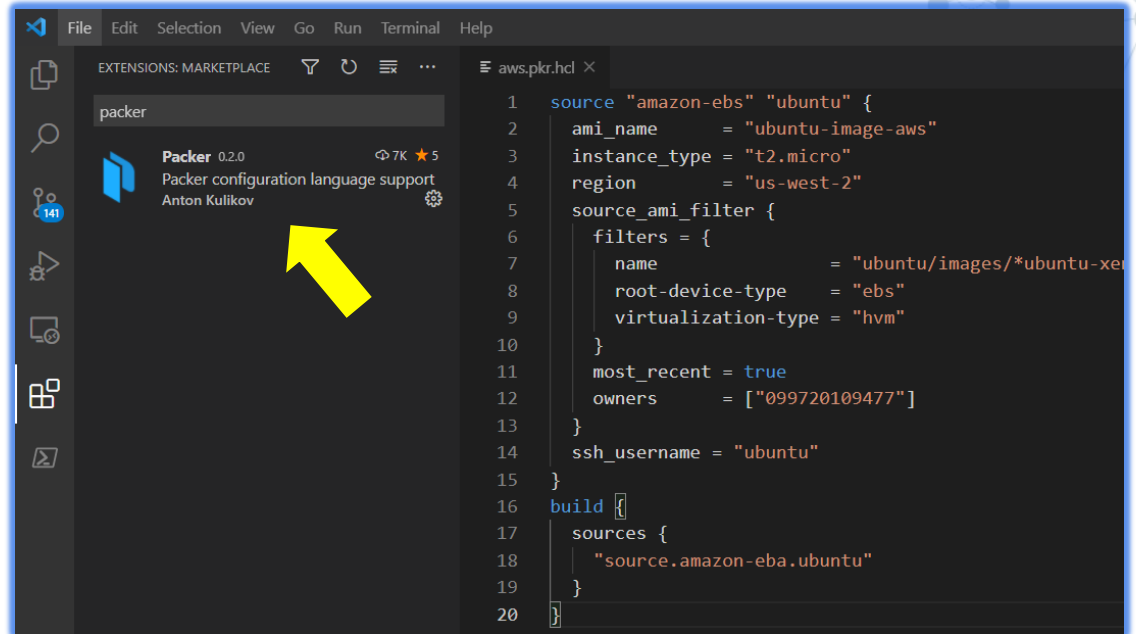
/* <-this is a multi-line comment
source "amazon-ecs" "example" {
  ami_name = "abc123"
}
*/
variable "example" {
```

Everything in green is commented out here



Syntax Highlighting

- Plugins for HCL exist for most major editors.
- VS Code Extension for Packer adds syntax support for the Packer HCL configuration language.



Interpolation Syntax

```
source "amazon-ebs" "amazon-ebs-amazonlinux-2" {
  ami_description      = "Vault - Amazon Linux 2"
  ami_name             = "vault-amazonlinux2"
  ami_regions         = ["us-east-1"]
  ami_virtualization_type = "hvm"
  associate_public_ip_address = true
  force_delete_snapshot = true
  force_deregister      = true
  instance_type         = "m5.large"
  region               = var.aws_region
  source_ami            = data.amazon-ami.amazon-linux-2.id
  tags = {
    Name      = "HashiCorp Vault"
    OS        = "Amazon Linux 2"
  }
  subnet_id      = var.subnet_id
  vpc_id         = var.vpc_id
}

build {
  sources = ["source.amazon-ebs.amazon-ebs-amazonlinux-2"]

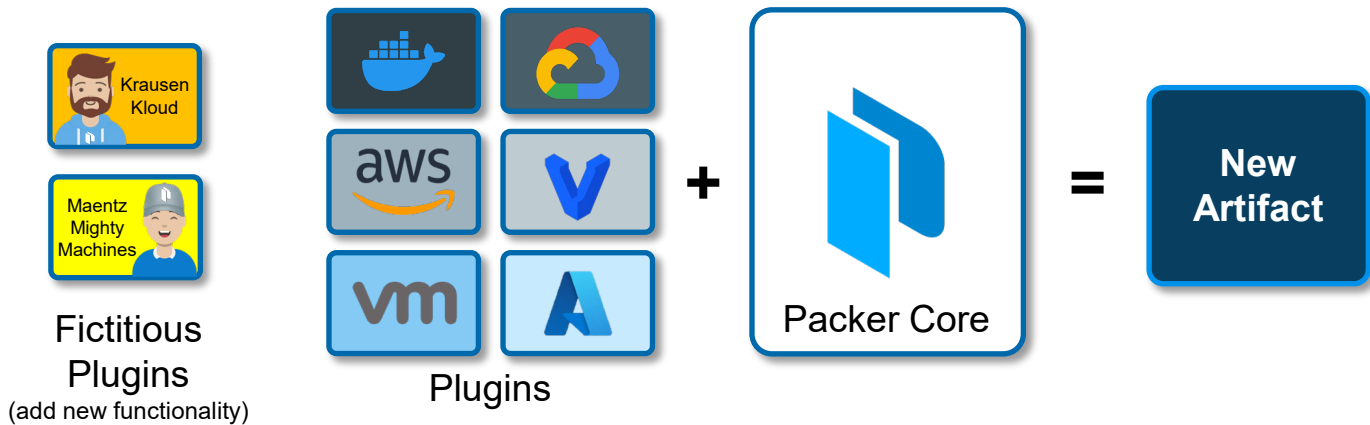
  provisioner "file" {
    destination = "/tmp/vault.zip"
    source      = var.vault_zip
  }
}
```

- Like Terraform, we can use interpolation syntax to refer to other blocks within the template
- Allows us to organize code as well as reuse values that are already defined or have been retrieved



Plugin Architecture

- Builders, provisioners, post-processors, and data sources are simply plugins that are consumed during the Packer build process
- This allows new functionality to be added to Packer without modifying the core source code

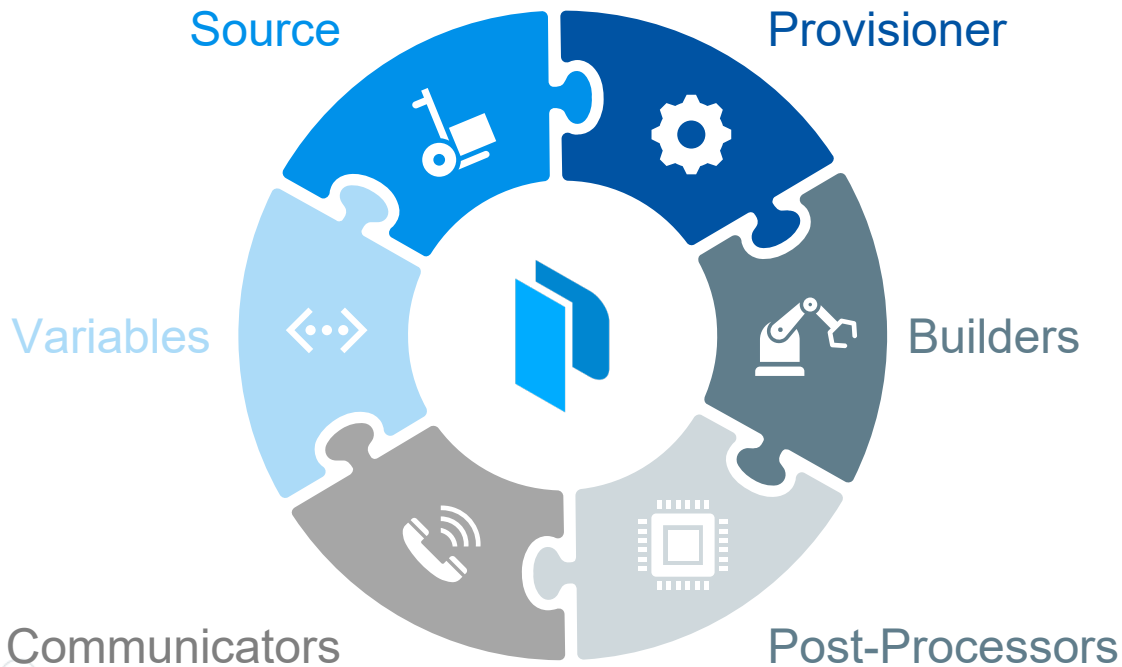




Working with **Packer's** Core Components in **HCL**



Core Components



Source Blocks

Only available in HCL templates



Source blocks define the initial image to use to create your customized image, how to launch the image, and how to connect to the image.

Sources can be used across multiple builds and references in builder blocks

```
source "amazon-ebs" "ubuntu" {  
  instance_type = "t2.micro"  
  region       = "us-west-2"  
  ami_id       = "ami-a1b2c3d4e5"  
}
```



Source Blocks

How to Launch Image & What Virtualization to Use

Source blocks define what kind of virtualization to use for the image, how to launch the image

In the example, the amazon-ebs builder configuration launches a t2.micro AMI in the us-west-2 region

```
amazon.pkr.hcl

source "amazon-ebs" "ubuntu" {
  ami_name      = "ubuntu-image-aws"
  instance_type = "t2.micro"
  region        = "us-west-2"
  source_ami_filter {
    filters = {
      name           = "ubuntu/images/*ubuntu-xenial-16.04-amd-server-*"
      root-device-type = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["099720109477"]
  }
}
```



Source Blocks

How to Connect to Image

Source blocks defining how to connect to the image.

In the example, the SSH communicator is used to allow Packer to SSH into the EC2 instance.

```
amazon.pkr.hcl

source "amazon-ebs" "ubuntu" {
  ami_name      = "ubuntu-image-aws"
  instance_type = "t2.micro"
  region        = "us-west-2"
  source_ami_filter {
    filters = {
      name           = "ubuntu/images/*ubuntu-xenial-16.04-amd-server-*"
      root-device-type = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["099720109477"]
  }
  ssh_username = "ubuntu"
}
```



Builders

Build blocks are used in tandem with source blocks and define what Packer should do with the image after it is launched.

The example template will be used to build an AWS Ubuntu AMI in the us-west-2 region. It references the AMI defined in the source block.

```
amazon.pkr.hcl

source "amazon-ebs" "ubuntu" {
  ami_name      = "ubuntu-image-aws"
  instance_type = "t2.micro"
  region        = "us-west-2"
  source_ami_filter {
    filters = {
      name           = "ubuntu/images/*ubuntu-xenial-16.04-amd-server-*"
      root-device-type = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["099720109477"]
  }
  ssh_username = "ubuntu"
}

build {
  sources {
    "source.amazon-ebs.ubuntu"
  }
}
```



Provisioners

Provisioners **install and configure** the machine image after it reboots. You would use one or many provisioners to customize the image as needed. Part of the build block.

File
Provisioner

Shell
Provisioners

```
build {  
  sources = ["sources.googlecompute.debian-build"]  
  
  provisioner "file" {  
    destination = "/tmp"  
    source      = "files"  
  }  
  provisioner "shell" {  
    script = "scripts/setup.sh"  
  }  
  provisioner "shell" {  
    inline = ["echo ${var.deployment_version} > ~/DEPLOYMENT_VERSION"]  
  }  
}
```

1st

2nd

3rd



Post-Processors

Post-processors can be used to specify what to do after the image is created. Part of the build block and is not mandatory.

Post
Processor



```
aws.pkr.hcl

build {
  sources {
    "source.amazon-ebs.ubuntu"
  }
  post-processor "shell-local" {
    inline = ["rm /tmp/script.sh"]
  }
}
```

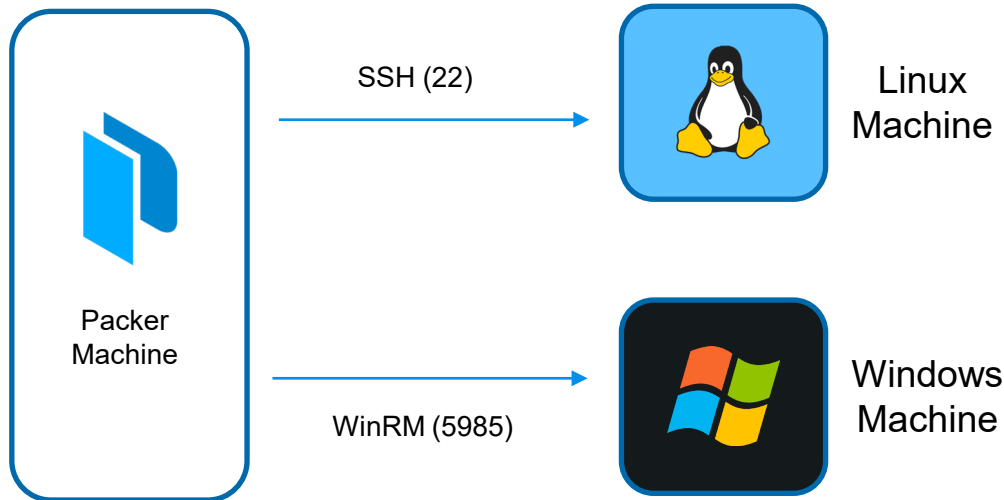


Communicators

Communicators are the mechanism that Packer uses to upload files and/or execute scripts when creating the final artifact.

Communicators are configured within the Source block of the Packer template.

- SSH - default
- WinRM
- Builder-Specific (docker exec)



Communicators

Linux

```
aws-centos.pkr.hcl

source "amazon-ebs" "centos" {
  ami_name      = "packer-centos-aws-{{timestamp}}"
  instance_type = "t2.micro"
  region        = "us-west-2"
  ami_regions   = ["us-west-2"]
  source_ami_filter {
    filters = {
      name                = "CentOS Linux 7 x86_64 HVM EBS *"
      product-code        = "aw0evgkw8e5c1q413zgy5pjce"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["679593333241"]
  }
  ssh_username = "centos"
}
```



SSH is the default Communicator
so we don't need to specify the
communicator type here



Communicators

Windows

Communicator

Communicator
Configuration

```
aws-windows.pkr.hcl

source "amazon-ebs" "windows-2012r2" {
  ami_name      = "my-windows-2012-aws-{{timestamp}}"
  communicator   = "winrm"
  instance_type = "t2.micro"
  region        = "us-east-1"
  source_ami     = "${data.amazon-ami.windows_2012r2.id}"
  user_data_file = "./scripts/SetUpWinRM.ps1"
  winrm_insecure = true
  winrm_use_ssl  = true
  winrm_username = "Administrator"
  tags = {
    "Name"           = "MyWindowsImage"
    "Environment"    = "Production"
    "OS_Version"     = "Windows"
    "Release"        = "Latest"
    "Created-by"     = "Packer"
  }
}
```



Remember that Windows is
not the default Communicator
so it must be defined



Variables

You can use input variables (aka *variables*) or local variables (aka *locals*) to define values for arguments throughout a Packer template.

variable block

variable block

```
aws.pkr.hcl

variable "subnet_id" {
  type    = string
  default = "subnet-1a2b3c4d5e"
}

variable "region" {
  type    = string
  default = "us-east-1"
}

source "amazon-ebs" "amazon-linux2" {
  ami_name      = local.ami_name
  instance_type = "t3.medium"
  region        = var.region
  source_ami     = data.amazon-ami.amazon-linux.id
  ssh_username   = var.ssh_username
  subnet_id     = var.subnet_id
  tags = {
    Name = local.ami_name
  }
  vpc_id = var.vpc_id
}
```

Defined in Variable Block

Referenced in Source Block





**END OF
SECTION**





Builders



Builders

- Builders are responsible for **creating machines and generating images** from them for various platforms
- You can specify **one or more** builder blocks in a template.
- Each builder block can **reference** one or more source blocks.
- There are many configuration options available for a given builder. Some options are required, and others are optional. The optional are dependent on the what the builder type supports.

Popular Builders

- | | |
|--|---------------------------------|
| ✓ AWS AMI Builder | ✓ Docker Builder |
| ✓ Azure Resource Manager Builder | ✓ Google Compute Builder |
| ✓ VMware Builder from ISO | ✓ Null Builder |
| ✓ VMware vSphere Clone Builder | ✓ QEMU Builder |
| ✓ VMware vSphere Builder from ISO | ✓ Virtual Box Builder |



Builders

```
azure.pkr.hcl

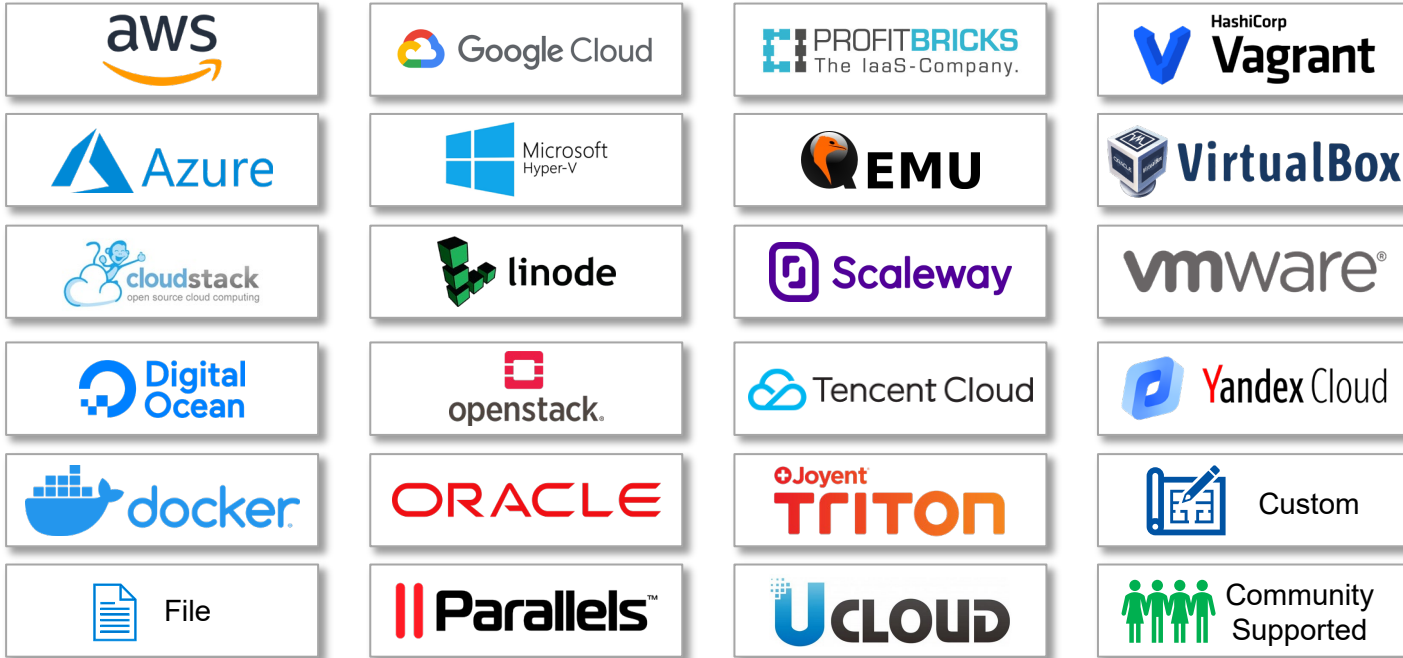
source "azure-arm" "azure-arm-centos-7" {
  image_offer      = "CentOS"
  image_publisher  = "OpenLogic"
  image_sku        = "7.7"
  os_type          = "Linux"
  subscription_id  = "${var.azure_subscription_id}"
}
build {
  source = ["source.azure-arm.azure-arm-centos-7"]

  provisioner "file" {
    destination = "/tmp/package_a.zip"
    source      = "${var.package_a_zip}"
  }
  provisioner "file" {
    destination = "/tmp/package_b.zip"
    source      = "${var.package_b_zip}"
  }
}
```

Uses the Azure builder to create a new Azure Machine Image



Builders



*Not an Exhaustive List





Examples of Packer Builders



Builders



```
source "amazon-ebs" "amazon-linux-2" {
  ami_name      = var.ami_name
  instance_type = "t3.medium"
  region        = var.region
  source_ami_filter {
    filters = {
      name              = var.source_ami_name
      root-device-type  = "ebs"
      virtualization-type = "hvm"
    }
    owners = [var.source_ami_owner]
  }
  ssh_username = var.ssh_username
  subnet_id    = var.subnet_id
  tags = {
    Name = var.ami_name
  }
  vpc_id = var.vpc_id
}
}
build {
  sources = ["source.amazon-ebs.amazon-linux-2"]

  provisioner "file" {
    destination = "/tmp"
    source      = "files"
  }
  provisioner "shell" {
    script = "scripts/setup.sh"
  }
}
```

amazon.pkr.hcl



Builders



ARM Builder

```
source "azure-arm" "azure-arm-centos-7" {
  azure_tags = {
    App-Version = var.app_version
    OS          = "centos"
    OS-Version  = "7"
    Owner       = var.owner
  }
  image_offer           = "CentOS"
  image_publisher       = "OpenLogic"
  image_sku             = "7.7"
  location              = var.azure_region
  managed_image_name    = "is-azure-immutable-vault-centos7"
  managed_image_resource_group_name = var.azure_resource_group_name
  os_type               = "Linux"
  subscription_id       = var.azure_subscription_id
  vm_size               = "Standard_D2_v3"
}
build {
  sources = ["source.azure-arm.azure-arm-centos-7"]
  provisioner "file" {
    destination = "/tmp/app.zip"
    source      = var.app_zip
  }
}
```



Builders



Google Cloud Builder

```
gcp.pkr.hcl

source "googlecompute" "debian-build" {
  project_id = "my project"
  source_image = "debian-9-stretch-v20200805"
  ssh_username = "packer"
  zone = "us-central1-a"
}

build {
  sources = ["sources.googlecompute.debian-build"]

  provisioner "shell" {
    script = "scripts/setup.sh"
  }
}
```



Builders



VMware ISO Builder

```
source "vmware-iso" "vmware-ubuntu" {  
  iso_url      = "http://ubuntu.com/releases/ubuntu-20.04-amd64.iso"  
  iso_checksum = "md5:af5f788aee1b32c4b2634734309cc9e9"  
  ssh_username = "packer"  
  ssh_password = "packer"  
  shutdown_command = "shutdown -P now"  
}  
  
build {  
  sources = ["sources.vmware-iso. vmware-ubuntu"]  
}
```



Builders



vSphere Clone Builder

```
ubuntu.pkr.hcl

source "vsphere-iso" "ubuntu" {
  CPUs           = "${var.vm-cpu}"
  RAM            = "${var.vm-mem}"
  cluster        = "${var.cluster}"
  convert_to_template = "true"
  datacenter     = "${var.datacenter}"
  datastore      = "${var.datastore}"
  disk_controller_type = "pvscsi"
  disk_size      = "${var.vm-disk-size}"
  disk_thin_provisioned = true
  folder         = "${var.folder}"
  vcenter_server = "${var.vcenter-server}"
  vm_name        = "${var.vm-name}"
}

build {
  sources = ["source.ubuntu"]

  provisioner "shell" {
    inline = ["echo 'Packer is Awesome'"]
  }
}
```





END OF SECTION





Packer Variables



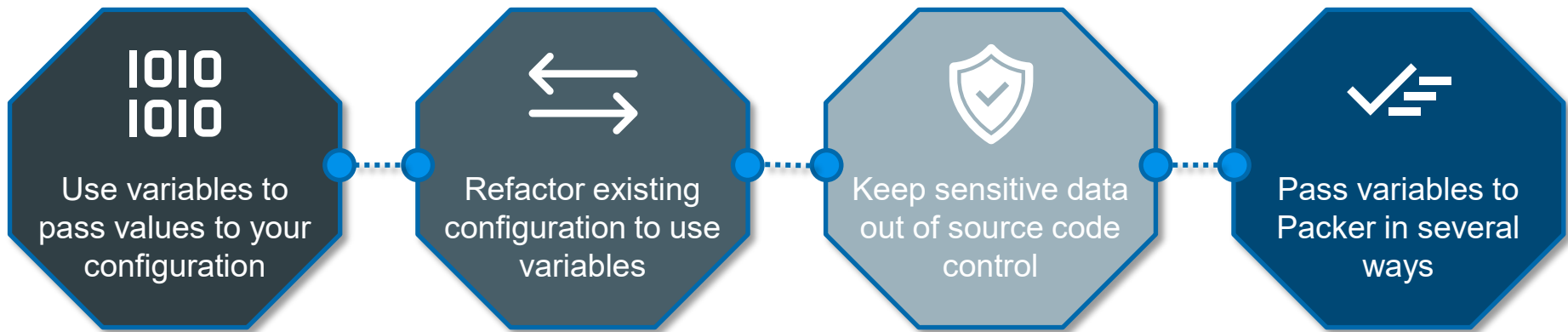
Introduction to Variables

- HashiCorp Packer can use **variables** to define defaults and values during a build
- Work a lot like variables from programming languages
- Allow you to **remove hard coded values** and pass parameters to your configuration
- Can help make configuration **easier to understand** and increase reusability
- Must always have a value. Variables are never optional, but they can have a default value

```
variable "image_id" {  
  type      = string  
  description = "The id of the machine image (AMI) to use for the server."  
  default   = "ami-1234abcd"  
  
  validation {  
    condition     = length(var.image_id) > 4 && substr(var.image_id, 0, 4) == "ami-"  
    error_message = "The image_id value must be a valid AMI id, starting with \"ami-\"."  
  }  
}
```



What can we do with **Packer** Variables?



Declaring Variables



- ◀ Variables can be declared and defined in a `.pkrrvars.hcl` file or `.auto.pkrrvars.hcl`, the default `.pkr` file, or any other file name if referenced when executing the build.



- ◀ You can also declare individually using the `-var` option



- ◀ Declared variables can be access throughout the template where needed within expressions



- ◀ The type is a constraint, meaning that's the only value that will be accepted for that variable



Understanding Variable Types

- The most common variable types are `string`, `number`, `list`, and `map`
- Other support types include `bool` (true/false), `set`, `object`, and `tuple`
- If type is omitted, it is inferred from the default value
- If neither type nor default is provided, the type is assumed to be `string`
- You can also specify complex types, such as collections

Simple Variable

```
variable "image_id" {  
  type      = string  
  description = "The id of the machine image (AMI)."  
  default   = "ami-1234abcd"  
}
```

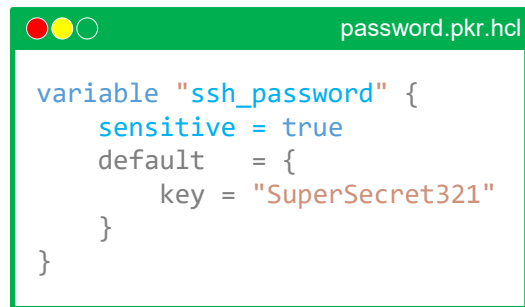
Variable Definition - Collection

```
variable "image_id" {  
  type      = list(string)  
  description = "The id of the machine image (AMI)."  
  default   = ["ami-1234abcd", "ami-1z2y3x4w5v"]  
}
```

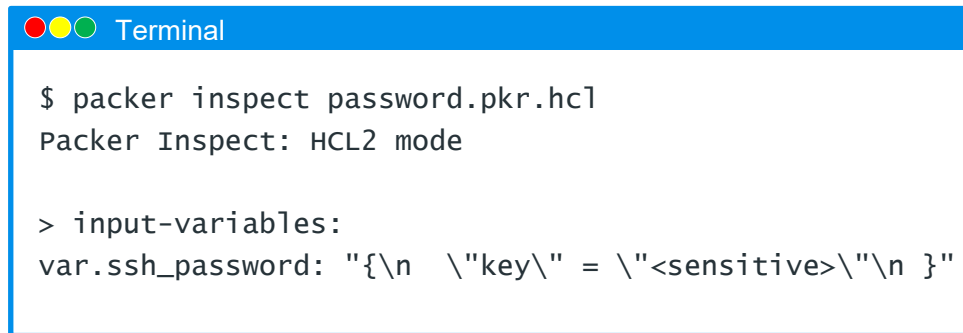
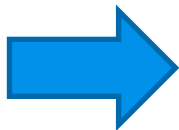


Sensitive Variables

- A variable can be marked as sensitive, if needed
- Packer will obfuscate the value in Packer's output



```
variable "ssh_password" {  
  sensitive = true  
  default   = {  
    key = "SuperSecret321"  
  }  
}
```



```
Terminal  
$ packer inspect password.pkr.hcl  
Packer Inspect: HCL2 mode  
  
> input-variables:  
var.ssh_password: "{\n  \"key\" = \"<sensitive>\"\n }"
```



Using Variables in Packer within Expressions



General Referral in Packer

`var.<name>`

Interpolation within a String

`"${var.<name>}"`



Using Variables in Packer within Expressions



General Referral
in Packer

Interpolation within
a String



Variable Use - General

```
image_name = var.image_name  
subnet_id  = var.subnet  
vpc_id     = var.vpc
```



Variable Use - String

```
image_name = "${var.prefix}-image-{{timestamp}}"
```



Variables

- Declared variables can be accessed throughout the template where needed
- Reference variables using expressions such as `var.<name>` or `"${var.<name>}"`

```
source "amazon-ebs" "aws-example" {  
  ami_name      = "aws-${var.ami_name}"  
  instance_type = "t3.medium"  
  region        = var.region  
  source_ami_filter {  
    filters = {  
      name           = var.source_ami_name  
      root-device-type = "ebs"  
      virtualization-type = "hvm"  
    }  
    owners = [var.source_ami_owner]  
  }  
}
```

ami_name variable reference

source_ami_name variable reference



Variables

Setting a Value for a Variable

```
aws-build.pkr.hcl

variable "image_id" {
  type      = string
  description = "The id of the machine image (AMI)"
  default = "ami-1234abcd"
}
```



What if we didn't want to use the default value of **ami-1234abcd**?

Options to Set Value for Variable:

```
variables.pkrvars.hcl

image_id = "ami-5678wxyz"
```

← Define the value in another file

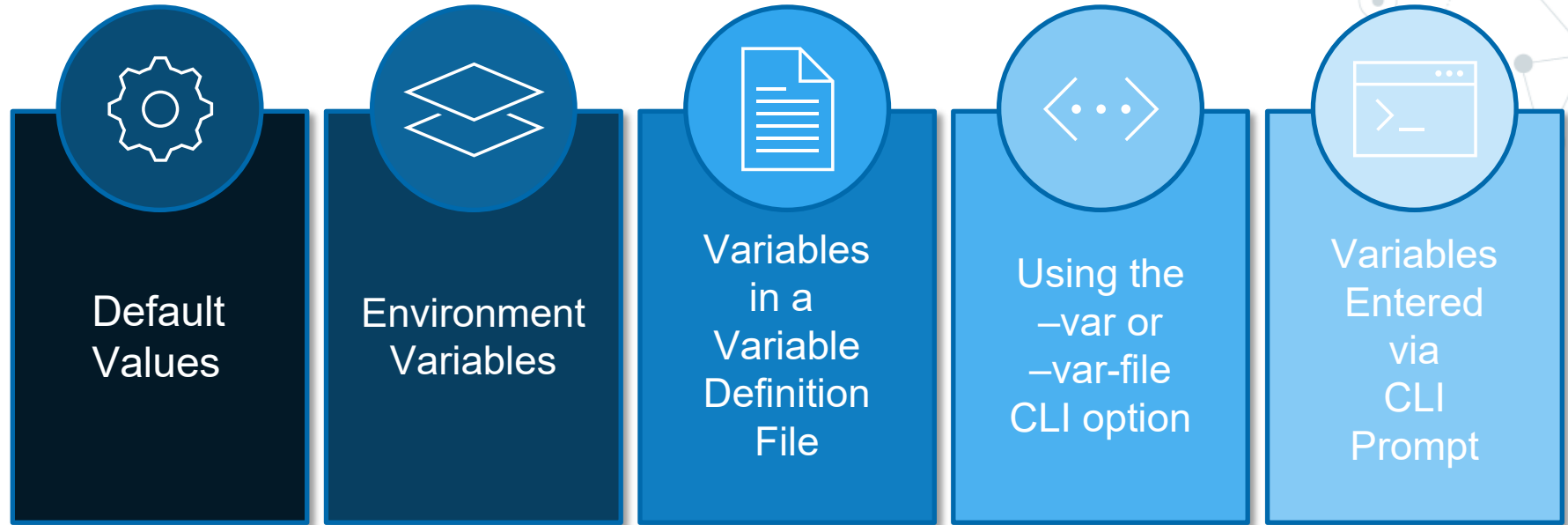
Declare the value on the command line during build →

```
Terminal

$ packer build -var image_id=ami-5678wxyz aws-build.pkr.hcl
```



Specifying Variable Values (Order of Operation)



Lowest Priority

Highest Priority



Locals

- Similar to input variables, assign a name to an expression or value
- Locals cannot be overridden at runtime – they are **constants**
 - Can use a `local { }` or `locals { }` block – can mark `local` as sensitive
 - Using `locals { }` is more compact and efficient
- Referenced in a Packer file through interpolation – `local.<name>`

```
aws-build.pkr.hcl

locals {
  timestamp = regex_replace(timestamp(), "[- TZ:]", "")
}

variable "image_name" {
  image_name = "${var.ami_prefix}-${local.timestamp}"
}
```

Define a local variable

Reference a local variable



Environment Variables

- Variables can also be set using [environment variables](#)
- Great solution for setting credentials or variables that might change often
- Packer will read environment variables in the form of `PKR_VAR_<name>`

```
Terminal  
  
$ export PKR_VAR_secret_key=AKIAIOSFODNN7EXAMPLE  
$ export PKR_VAR_access_key=wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY  
  
$ packer build aws-linux.pkr.hcl
```

Set environment variables

Run Packer build that will use ENV



Default Environment Variables

- Packer will automatically define certain commonly used environment variables at build time that can be referenced

`PACKER_BUILD_NAME` – set to the name of the build that Packer is running.

`PACKER_BUILD_TYPE` – set to the type of builder that was used to create the machine

`PACKER_HTTP_ADDR` – set to the address of the http server for file transfer (if used)





**END OF
SECTION**



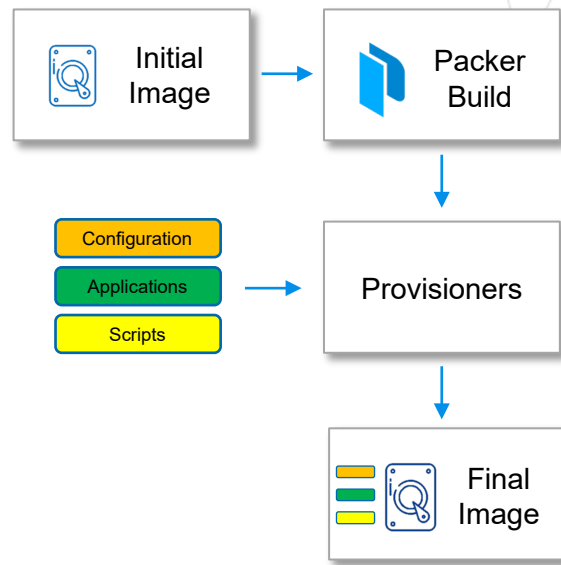


Provisioners



Introduction to Provisioners

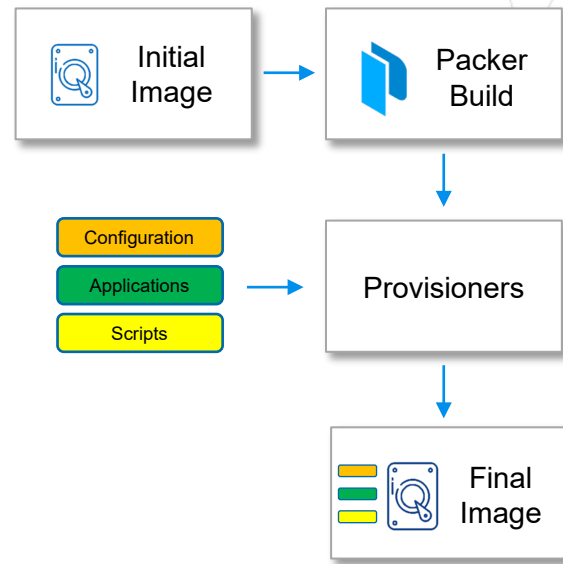
- Provisioners use built-in and third-party integrations to **install packages** and **configure the machine image**
- Built-in integrations include **file** and different **shell** options
- Third-party integrations include:
 - **Ansible** – run playbooks
 - **Chef** – run cookbooks
 - **InSpec** – run InSpec profiles
 - **PowerShell** – execute PowerShell scripts
 - **Puppet** – run Puppet manifest
 - **Salt** – configure based on Salt state
 - **Windows Shell** – runs commands using Windows cmd



Introduction to Provisioners

Provisioners prepare the system for use, so common use cases for provisioners include:

- installing packages
- patching the kernel
- creating users
- downloading application code



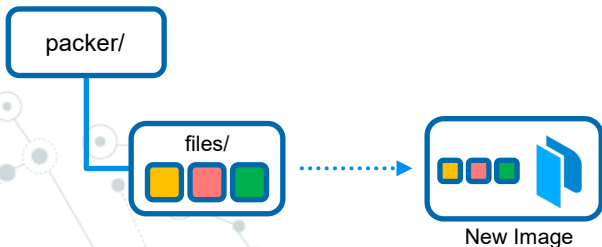
Examples of Provisioners

Upload a Single File

```
provisioner "file" {  
  source = "packer.zip"  
  destination = "/tmp/packer.zip"  
}
```

Upload Multiple Files in a Local Directory

```
provisioner "file" {  
  source      = "/files"  
  destination = "/tmp"  
}
```



Examples of Provisioners

Execute a Script

```
provisioner "shell" {  
  script = "install_something.sh"  
}
```

Run Commands on the Machine

```
provisioner "shell" {  
  inline = [  
    "echo Installing Updates",  
    "sudo apt-get update",  
    "sudo apt-get install -y nginx"  
  ]  
}
```



Examples of Provisioners

Run an Ansible Playbook

```
provisioner "ansible" {  
  ansible_env_vars = ["ANSIBLE_HOST_KEY_CHECKING=False"]  
  extra_arguments  = ["--extra-vars", "desktop=false"]  
  playbook_file    = "${path.root}/playbooks/playbook.yml"  
  user             = var.ssh_username  
}
```

Execute a PowerShell script

```
provisioner "powershell" {  
  script = [".scripts/win2019.ps1"]  
}
```



Useful Features of Provisioners



Run on Specific Builds

- Use `only` and `except` to run provisioner with specific builds
- This was shown in Target Cloud and Build Types demo (builders section)

Build-Specific Overrides

- Can override builds if they behave differently in order to achieve similar results
- Could be used if building images across different platforms so you end up with a like-for-like images
- **Example:** Admin access or agent installs

```
provisioner "shell-local" {  
  inline = ["/tmp/install_vmware-tools.sh"]  
  override = {  
    aws = {  
      inline = ["/tmp/install_cloudwatch_agent.sh"]  
    }  
  }  
}
```



Useful Features of Provisioners



Using the On Error Provisioner

- Special provisioner that only runs if the related-provisioner fails
- Runs before the instance is shutdown/terminated
- Write data to a file, unsubscribe from a service, clean up custom work

```
build {  
  sources = ["aws-amazonlinux-build"]  
  provisioner "shell-local" {  
    inline = ["sudo yum update -y"]  
  }  
  error-cleanup-provisioner "shell-local" {  
    inline = ["echo 'update provisioner failed'> packer_log.txt"]  
  }  
}
```

Only runs if other
provisioner fails



Useful Features of Provisioners



Introducing a Pause

- Introduce a waiting period before running
- Helpful when it takes a bit for the OS to come up, or other processes are running that could conflict with provisioner
- Uses the `pause_before` value. By default, there is no pause.

```
build {  
  sources = ["aws-ubuntu-build"]  
  provisioner "shell-local" {  
    inline = ["sudo yum update -y"]  
    pause_before = "10s"  
  }  
}
```

Wait 10 seconds before executing



Useful Features of Provisioners



Retries

- Instruct a provisioner to try again if it fails
- Helpful if a provisioner depends on external data/processes to complete successfully
- Uses the `max_retries` value. By default, this value is `0` and there is no retry

```
build {  
  sources = ["aws-ubuntu-build"]  
  provisioner "shell-local" {  
    inline = ["sudo yum update -y"]  
    max_retries = 5  
  }  
}
```

Try again (up to 5 times)



Useful Features of Provisioners



Timeouts

- Restrict the time the provisioner should wait until you consider it failed
- Uses the `timeout` value. By default, there is no timeout

```
build {  
  sources = ["aws-ubuntu-build"]  
  provisioner "shell-local" {  
    inline = ["../install_something.sh"]  
    timeout = "5m"  
  }  
}
```

Wait 5 minutes before marking it as failed





**END OF
SECTION**

